

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Alya Hasna Fadilah- 5024241044

2026

1 Pendahuluan

1.1 Latar Belakang

Perkembangan teknologi elektronika dan sistem tertanam (embedded system) telah mendorong pemanfaatan mikrokontroler dalam berbagai perangkat elektronik modern. Mikrokontroler berperan sebagai pusat kendali yang mampu menerima masukan, mengolah data berdasarkan program, serta menghasilkan keluaran untuk mengendalikan perangkat elektronik. Penerapannya dapat ditemukan pada sistem kendali otomatis, perangkat berbasis Internet of Things (IoT), dan sistem pemantauan lingkungan.

Dalam penggunaannya, terdapat beberapa permasalahan yang perlu dipahami, seperti ketidakstabilan pembacaan sinyal akibat kondisi floating, kesalahan pendeteksian tombol akibat fenomena bouncing, serta perbedaan karakteristik dan metode pemrograman pada setiap platform mikrokontroler. Selain itu, mikrokontroler membutuhkan kemampuan untuk membaca perubahan besaran fisik melalui sensor dengan sinyal analog. Permasalahan tersebut menjadi dasar pentingnya mempelajari konfigurasi GPIO, pengolahan sinyal digital, serta konversi sinyal analog menjadi data digital.

Praktikum bertujuan agar kita dapat mempelajari penggunaan STM32 dan ESP32 sehingga kita dapat mengetahui pengendalian LED sebagai digital output, pembacaan pushbutton sebagai digital input menggunakan resistor pull-up dan pull-down, penerapan teknik debouncing, serta pembacaan intensitas cahaya menggunakan sensor LDR dan ADC. Melalui praktikum ini, praktikan diharapkan mampu memahami prinsip dasar pengoperasian mikrokontroler dan mengimplementasikannya sebagai landasan pengembangan sistem tertanam yang dapat diterapkan pada berbagai teknologi modern.

1.2 Dasar Teori

1.2.1 PlatformIO dan VSCode sebagai Lingkungan Pengembangan

PlatformIO adalah ekosistem pengembangan embedded lintas platform yang mendukung ratusan board dan berbagai framework (Arduino, STM32Cube/HAL, ESP IDF, dan lainnya) melalui satu antarmuka yang seragam di dalam Visual Studio Code. Keunggulan utamanya adalah kemampuan menjalankan framework yang berbeda pada chip yang sama hanya dengan mengubah berkas konfigurasi. Sebagai contoh, ESP32 dapat diprogram dengan framework Arduino yang sederhana maupun ESP IDF yang lebih native.

Struktur proyek PlatformIO terdiri atas empat bagian:

- platformio ini: konfigurasi board, framework, dan protokol upload.
- src/: kode program utama.
- include/: berkas header.
- lib/: library tambahan.

1.3 STM32 Blackpill dan Framework Arduino

Blackpill adalah *development board* berbasis mikrokontroler STM32F401CC, yaitu prosesor 32-bit ARM Cortex-M4. *Board* ini memiliki memori Flash 256 KB dan RAM 64 KB. LED *onboard* terhubung

ke pin PC13 dengan logika aktif LOW, sedangkan tombol pengguna berada di pin PA0. *Board* ini tidak memiliki *programmer* bawaan, sehingga pemrograman memerlukan **ST-Link** eksternal melalui protokol ***Serial Wire Debug* (SWD)** yang memakai jalur SWDIO, SWCLK, GND, dan 3.3V.

STM32 dapat diprogram dengan *framework* Arduino yang dikenal sebagai **STM32duino**. *Framework* ini menyediakan fungsi seperti `pinMode()`, `digitalWrite()`, dan `delay()`, sehingga pemula tidak perlu langsung mempelajari register atau HAL.

1.4 ESP32 dan Framework ESP-IDF

ESP32 adalah mikrokontroler 32-bit *dual-core* (Xtensa LX6) dengan WiFi dan Bluetooth terintegrasi. *Board* ESP32 DevKit dilengkapi *USB-to-Serial* bawaan sehingga dapat diprogram langsung lewat kabel USB tanpa *programmer* eksternal.

ESP-IDF (*Espressif IoT Development Framework*) adalah *framework* resmi dan *native* dari Espressif yang dibangun di atas sistem operasi waktu nyata **FreeRTOS**. Program ESP-IDF memakai struktur berbasis *task*/*component* dengan titik masuk berupa fungsi `app_main()`, berbeda dengan Arduino yang menyederhanakan program menjadi `setup()` dan `loop()`. ESP-IDF memberi akses lebih penuh ke fitur ESP32, seperti konfigurasi WiFi tingkat rendah dan penjadwalan *task* FreeRTOS secara langsung, sehingga umum dipakai untuk pengembangan produk IoT tingkat lanjut.

Fungsi `vTaskDelay()` pada ESP-IDF bekerja berdasarkan *tick* FreeRTOS dan tidak memblokir *task* lain, sedangkan `delay()` pada Arduino bersifat menunda eksekusi program secara umum.

1.5 GPIO dan Konfigurasi *Pull-up*/*Pull-down* pada Tombol

General Purpose Input/Output (GPIO) adalah pin serbaguna pada mikrokontroler yang dapat dikonfigurasi sebagai keluaran (*output*) untuk mengendalikan perangkat seperti LED, atau masukan (*input*) untuk membaca kondisi eksternal seperti tombol.

Ketika tombol tidak ditekan dan pin GPIO tidak terhubung ke tegangan mana pun, pin berada dalam kondisi ***floating***. Nilai yang terbaca tidak pasti, bisa HIGH atau LOW secara acak akibat *noise*. Untuk mengatasinya, pin perlu "ditarik" ke level tegangan tertentu menggunakan resistor:

- ***Pull-up resistor*** menghubungkan pin GPIO ke VCC (3.3V) melalui resistor, umumnya 10 kΩ. Saat tombol tidak ditekan, pin terbaca **HIGH**. Saat tombol ditekan dan menghubungkan pin ke GND, pin terbaca **LOW**.
- ***Pull-down resistor*** menghubungkan pin GPIO ke GND melalui resistor. Saat tombol tidak ditekan, pin terbaca **LOW**. Saat tombol ditekan dan menghubungkan pin ke VCC, pin terbaca **HIGH**. Logika ini berkebalikan dengan *pull-up*.

Resistor ini dapat dipasang secara **eksternal** (komponen fisik pada *breadboard*) atau diaktifkan secara **internal** melalui *firmware* tanpa komponen tambahan. Pada ESP32 dengan *framework* Arduino, *pull-up* internal diaktifkan dengan `pinMode(pin, INPUT_PULLUP)` dan *pull-down* internal dengan `pinMode(pin, INPUT_PULLDOWN)`. Perlu diperhatikan bahwa tidak semua pin GPIO ESP32 mendukung resistor *pull* internal. Pin *input-only* (GPIO 34–39) tidak memilikinya sama sekali, sehingga wajib memakai resistor eksternal jika dipakai sebagai input tombol.

1.6 *Debouncing*

Kontak mekanik pada tombol atau saklar menghasilkan beberapa transisi sinyal HIGH–LOW dalam waktu sangat singkat akibat getaran fisik saat kontak bersentuhan atau terlepas. Fenomena ini disebut ***bouncing***. Tanpa penanganan, satu kali penekanan dapat terbaca sebagai beberapa kejadian sekaligus.

Debouncing memastikan hanya satu transisi valid yang terdeteksi. Pada pendekatan perangkat lunak, program menunggu sinyal stabil selama periode tertentu, misalnya 20–50 ms, sebelum menganggap perubahan tersebut sah. Implementasinya memakai fungsi `millis()` untuk mencatat waktu terakhir sinyal berubah. Status tombol baru diperbarui jika sinyal tidak berubah selama lebih dari batas waktu *debounce*. Cara ini bersifat *non-blocking* karena tidak menghentikan jalannya program seperti `delay()`.

1.7 ADC sebagai Mode Akses GPIO Ketiga

GPIO dapat diakses dalam tiga mode: *digital output* (menyalakan LED), *digital input* (membaca tombol, hanya mengenal HIGH atau LOW), dan ***analog input***, yaitu membaca tegangan yang berubah secara kontinu melalui periferal **ADC (*Analog-to-Digital Converter*)**.

ADC mengubah tegangan analog pada suatu pin menjadi nilai digital yang dapat diolah program. ESP32 memiliki ADC internal dengan resolusi *default* **12-bit**, sehingga menghasilkan nilai 0–4095, dengan referensi tegangan sekitar 0–3.3V. Pembacaannya dilakukan dengan fungsi `analogRead(pin)` pada *framework* Arduino. Nilai digital dikonversi kembali menjadi perkiraan tegangan dengan persamaan:

$$V = \text{raw} \times \frac{3.3}{4095} \tag{1}$$

Tidak semua pin GPIO mendukung ADC. Pada ESP32, pin ADC1 (misalnya GPIO 32–39) lebih disarankan daripada ADC2, karena ADC2 memiliki keterbatasan ketika modul WiFi aktif.

1.8 LDR dan Rangkaian Pembagi Tegangan

Light Dependent Resistor **(LDR)** adalah resistor yang nilai hambatannya berubah menurut intensitas cahaya yang diterimanya. Hambatannya tinggi saat gelap dan menurun saat terang. Karena ADC hanya membaca tegangan, perubahan hambatan ini diubah menjadi perubahan tegangan melalui rangkaian **pembagi tegangan** (*voltage divider*) yang memasangkan LDR dengan resistor tetap (10 kΩ pada praktikum ini):

$$V_o = V_{in} \times \frac{R_1}{R_{LDR} + R_1} \tag{2}$$

2 Tugas Pendahuluan

Bagian ini berisi jawaban dari tugas pendahuluan yang telah anda kerjakan, beserta penjelasan dari jawaban tersebut

1. Arduino menggunakan framework dengan fungsi-fungsi sederhana jadi lebih mudah digunakan dan dapat memprogram mikrokontroler tanpa harus memahami seluruh detail. Kelebihannya adalah sintaks yang mudah dipahami, banyaknya library, dan sering digunakan oleh umum.

Kekurangannya adalah keterbatasan dalam pengaturan fitur perangkat keras secara mendalam dan kurang fleksibel untuk pengembangan sistem yang kompleks. Sedangkan, ESP-IDF (Espressif IoT Development Framework) dirancang khusus untuk pengembangan mikrokontroler ESP32. Jadi, ESP-IDF mempunyai akses lebih luas terhadap fitur perangkat keras, seperti GPIO, Wi-Fi, Bluetooth, timer, dan multitasking menggunakan FreeRTOS. Kelebihannya adalah fleksibel dan kontrol yang lebih tinggi terhadap sistem. Kekurangannya adalah penggunaannya lebih kompleks dan membutuhkan pemahaman pemrograman yang lebih mendalam.

2. Karena STM32 Blackpill memerlukan programmer eksternal seperti ST-Link karena board tersebut tidak selalu dilengkapi dengan chip USB-to-Serial atau USB programming. , sehingga program dapat ditulis langsung ke memori Flash mikrokontroler. Kalau sESP32 Development Kit memiliki chip USB-to-Serial, seperti CP2102 atau CH340. Chip tersebut berfungsi mengubah komunikasi USB dari komputer menjadi komunikasi UART yang dapat dipahami oleh ESP32.

3. Pull-up resistor menghubungkan pin input ke sumber tegangan positif (VCC), sehingga kondisi awal pin adalah HIGH. Ketika tombol ditekan, pin akan terhubung ke GND sehingga kondisinya berubah menjadi LOW.

   Sedangkan Pull-down resistor menghubungkan pin input ke GND, sehingga kondisi awal pin adalah LOW. Ketika tombol ditekan, pin akan terhubung ke VCC sehingga kondisinya berubah menjadi HIGH.

4. Bouncing adalah ketika kontak logam pada tombol mekanik mengalami sentuhan dan pelepasan secara berulang dalam waktu yang sangat singkat ketika tombol ditekan atau dilepaskan. Akibatnya, mikrokontroler dapat membaca satu kali penekanan tombol sebagai beberapa perubahan kondisi logika sehingga Hal tersebut menyebabkan kesalahan pembacaan input, seperti penghitung yang bertambah beberapa kali hanya karena satu kali tombol ditekan. Pendekatan yang bisa dilakukan adalah

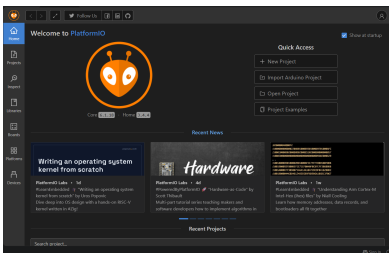
   - Software Debouncing : menggunakan program untuk memberikan jeda waktu setelah tombol ditekan
   - Hardware Debouncing : menggunakan komponen elektronik seperti kapasitor/ resistor untuk meredam perubahan sinyal saat tombol ditekan

5. ADC (Analog-to-Digital Converter) berfungsi mengubah sinyal analog menjadi nilai digital agar dapat diproses oleh mikrokontroler. prosesnya ada beberaoa tahapan yaitu :

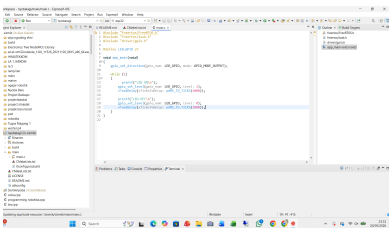
   - Sampling : mengambil sample tegangan analog
   - Quantization : mengelompokkan nilai tegangan hasil sampling
   - Encoding : mengubah hasil kuantisasi menjadi representasi bilangan biner yang dapat diproses oleh mikrokontroler

   Resolusi ADC ditentukan oleh jumlah bit yang digunakan. Semakin besar jumlah bit, semakin banyak tingkat nilai digital yang dapat direpresentasikan sehingga hasil pembacaan tegangan menjadi lebih detail.

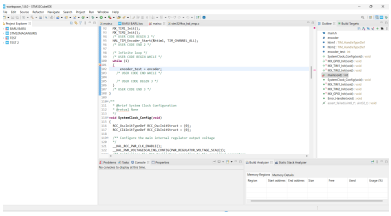
6. Setup tools VS Code, PlatformIO, ESP-IDF, STM32, dan Serial Plotter



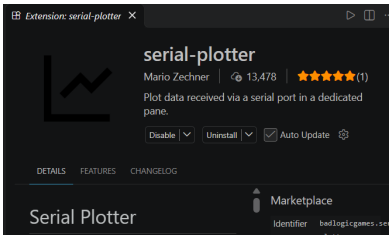
Gambar 1: Setup PlatformIO



Gambar 2: Setup ESP-IDF



Gambar 3: STM-32



Gambar 4: Serial Plotter